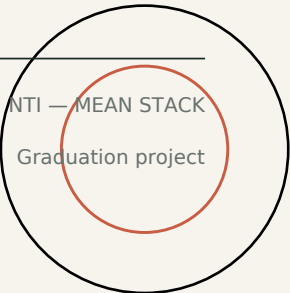


Dawwarha

Turning unused resources
into community impact.

A structured coordination layer for surplus and need



01 / ORIENTATION

A platform for the moment surplus meets need.

Dawwarha is a community resource-rescue platform that connects unused or surplus resources with the people, organizations, and communities that need them.

THE PROBLEM

Usable resources go to waste because supply and need are rarely visible to each other at the right time.

THE IDEA

Publish resources and requests in a structured system, then surface transparent, location- and urgency-aware matches.

THE OPPORTUNITY

Remove coordination friction without asking communities to become more generous than they already are.

The product thesis

When someone has a surplus resource and someone else has a genuine need for that same category of resource, Dawwarha should surface that match automatically, guide both sides through a safe handover, and record the outcome.

The pages that follow are the single source of truth for the product concept, MVP scope, technical direction, and path to a credible demonstration.

CONTENTS

A brief built to be used.

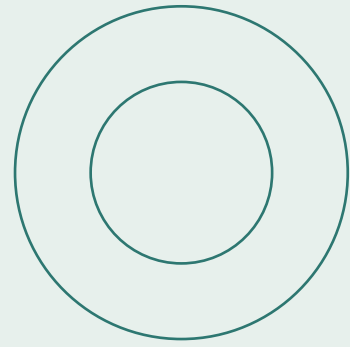
01 The problem	5
01 Executive Summary	6
02 Problem Definition	6
03 The opportunity	8
03 Opportunity	9
04 Product Vision	9
05 Core Product Thesis	9
06 The people	10
06 Target Users	11
07 User Jobs-To-Be-Done	11
08 The transfer loop	13
08 User Journey	14
09 Product Ecosystem	14
10 The product system	16
10 Core Features	17
11 Resource Lifecycle	17
12 Matching System	18
13 Trust & Safety	19
14 Roles & Authorization	20
15 CRUD Mapping	20
16 The contract	22
16 Functional Requirements	23
17 Non-Functional Requirements	23
18 The technical foundation	24
18 MEAN Architecture	25

19 Data Model	26
20 API Design	26
21 Validation Strategy	27
22 Error Handling	27
23 The experience	29
23 UX & UI Principles	30
24 Key Screens	30
25 Admin Dashboard	31
26 The measure	33
26 Impact Measurement	34
27 Business / Sustainability Model	34
28 Competitive Landscape	35
29 SWOT Analysis	35
30 Risks & Mitigation	36
31 The path to delivery	37
31 MVP Definition	38
32 Future Roadmap	38
33 Team Structure	38
34 Development Strategy	39
35 Testing Strategy	39
36 Definition of Done	40
37 Demo Strategy	40
38 Evaluation Strategy	41
39 Product Success Definition	41
40 Final Product Positioning	41

01

THE PROBLEM

The gap is not a lack of generosity — it is a lack of coordination infrastructure.



01 — Executive Summary

Dawwarha is a community resource-rescue platform that connects unused or surplus resources — food, books, furniture, clothing, school supplies, office equipment, and other legally transferable items — with the people, organizations, and communities that need them.

The problem: usable resources go to waste not because nobody needs them, but because the person who has a surplus and the person who has a need are rarely visible to each other at the right time. Today, this coordination happens informally through scattered Facebook posts, WhatsApp groups, and word of mouth — channels that were never designed for structured discovery, matching, or follow-through.

The solution: Dawwarha gives providers a place to publish what they have, gives seekers and organizations a place to publish what they need, and uses a transparent matching system to connect the two based on category, location, quantity, urgency, and availability. Every transfer moves through a defined lifecycle — from listing to match to handover to confirmation — so outcomes are trackable instead of anecdotal.

Who it serves: individuals and households with surplus items, individuals in need, and community organizations (NGOs, schools, community centers) that coordinate distribution at scale. An admin layer maintains trust and moderates the platform.

Why it matters: the underlying inefficiency — supply and demand for reusable resources being disconnected — is structural, not a matter of goodwill. A dedicated, structured coordination layer reduces waste, reduces friction, and gives visibility to needs that would otherwise go unmet.

What makes it different: Dawwarha is purpose-built for resource matching, not a repurposed general-purpose social feed. It combines structured listings, need publishing, a scoring-based matching engine, verification and moderation, and lifecycle tracking — capabilities that Facebook and WhatsApp groups do not offer.

Expected impact (Hypothesis): an increase in the number of surplus resources successfully redirected to people or organizations who need them, measured through platform-native metrics such as completed transfers and fulfilled requests, not through unverified environmental claims.

02 — Problem Definition

Problem Statement

People and organizations with surplus resources and people with unmet needs for those same resources frequently exist in the same community without a reliable way to find each other in time. The result is that usable resources are discarded, stored indefinitely, or transferred inefficiently through informal channels.

Root Causes

- 1 **Fragmented communication:** offers and requests are scattered across multiple Facebook groups, WhatsApp chats, and personal networks with no shared structure.
- 1 **No centralized coordination layer:** there is no single place to search "what's available near me" or "who needs what I have."
- 1 **Informal donation processes:** handovers are arranged ad hoc, with no confirmation step, no accountability, and no record of outcome.
- 1 **Time sensitivity:** many resources (especially food) have a narrow window of usefulness; by the time a post is seen, the opportunity has often passed.
- 1 **Transportation friction:** without location-aware matching, a potential match may be discovered but be impractical to fulfill.
- 1 **Trust gaps:** anonymous posts in open groups make it hard to know if a listing or request is genuine.

Current Friction

A typical Facebook donation post is a single unstructured message, broadcast once, with no persistent listing, no categorization, no way to filter by location or urgency, and no confirmation that the item was actually received.

Existing Alternatives

Channel	What it offers	What it lacks
Facebook groups	Reach, familiarity	No structure, no matching, no lifecycle
WhatsApp groups	Immediacy	No discoverability, no history, group-size limits
Traditional donation organizations	Trust, established process	Slow, centralized, not resource-specific, limited reach for small/local surplus
Classified platforms (e.g. OLX-style)	Structured listings	Built for buying/selling, not need-matching or community impact tracking

Gaps in Existing Solutions

None of the above combine: structured need publishing, location- and urgency-aware matching, a defined transfer lifecycle, and verifiable outcome tracking.

Why This Problem Matters

Every unmatched surplus resource represents both an avoidable loss and an unmet need that persists longer than necessary. The gap is not a lack of generosity — it is a lack of coordination infrastructure.

03

Make surplus visible, make needs legible, and reduce the friction between them.

03 — Opportunity

A digital platform is well-suited to this problem because the core failure is one of **information and coordination**, not motivation. Digital systems are efficient at:

- 1 Making unused resources **discoverable** by the people who need them.
- 1 Making community **needs visible** to people who have surplus but don't know who needs it.
- 1 Applying **structured matching logic** that a manual group feed cannot.
- 1 Establishing **trust mechanisms** (verification, reporting, reputation) that scattered informal channels lack.
- 1 Turning individually invisible small acts (one bag of clothes, one box of books) into a **measurable collective pattern** of community participation.

The opportunity is not to replace generosity, but to remove the friction that currently prevents generosity from reaching its destination.

04 — Product Vision

Vision Statement

A community where no usable resource goes to waste simply because the right person didn't know it existed.

Mission Statement

To give every community a structured, trustworthy way to move surplus resources to the people and organizations who need them.

Product Philosophy

Dawwarha treats resource redistribution as a coordination problem to be engineered, not a goodwill problem to be hoped for. The platform's job is to make the right connection visible, fast, and safe to act on.

Long-Term Ambition

To become the default coordination layer that individuals, schools, and community organizations reach for whenever a resource is unused on one side and needed on the other — starting with a single community, and expanding as trust and adoption compound.

05 — Core Product Thesis

When someone has a surplus resource and someone else has a genuine need for that same category of resource, Dawwarha should surface that match automatically, guide both sides through a safe handover, and record the outcome — resulting in less waste, less unmet need, and a visible record of community contribution.

06

Design around the people who give, seek, coordinate, and protect the system.

06 — Target Users

Persona A — Resource Provider

Profile: An individual, household, restaurant, shop, or small company with recurring or one-off surplus (unused furniture, extra food, outgrown clothing, retired office equipment). **Goals:** Get surplus items to someone who will actually use them, with minimal effort. **Pain points:** No easy way to reach the right recipient; unclear who to trust; no record that the item was actually used well. **Behaviors:** Currently posts in Facebook groups or asks around informally; often gives up and discards items when no quick match appears. **Needs:** A simple listing flow, visibility into interested recipients, a straightforward handover process. **Frustrations:** Repeated messages from non-serious inquiries; no way to prioritize genuine needs. **Desired outcome:** A confirmed transfer with minimal coordination overhead. **Platform interaction:** Publishes resources, reviews matches/requests, confirms handovers, sees contribution history.

Persona B — Resource Seeker

Profile: An individual who needs a specific item or category of item and cannot afford or easily access it through normal channels. **Goals:** Find a genuine, nearby, available resource quickly. **Pain points:** Scattered listings, no way to filter by what's actually relevant, uncertainty about whether an offer is still valid. **Behaviors:** Searches Facebook groups, messages multiple contacts, often finds outdated or already-claimed offers. **Needs:** A reliable way to publish a request, know its status, and be notified of a match. **Frustrations:** Wasted time chasing dead leads. **Desired outcome:** A confirmed, timely match that meets an actual need. **Platform interaction:** Publishes requests, browses resources, accepts matches, confirms receipt.

Persona C — Community Organization

Profile: An NGO, school, or community center that coordinates distribution of resources to multiple beneficiaries. **Goals:** Source resources reliably and at scale; maintain credibility with donors and beneficiaries. **Pain points:** Manual outreach for every campaign; no structured way to signal ongoing needs; verification of legitimacy is manual and slow. **Behaviors:** Runs periodic donation drives through personal networks and social media. **Needs:** Verified organizational status, the ability to publish standing needs, visibility into fulfillment history. **Frustrations:** Difficulty proving impact to their own stakeholders. **Desired outcome:** A steady, trackable pipeline of matched resources and a record of impact to report upward. **Platform interaction:** Registers and completes verification, publishes needs, manages incoming matches, reports impact.

Persona D — Platform Administrator

Profile: A team member (initially one of the 4 developers, later a dedicated moderator) responsible for platform integrity. **Goals:** Keep the platform trustworthy, functional, and free of abuse. **Pain points:** Fake listings, spam, disputes between users. **Behaviors:** Reviews reports, verifies organizations, monitors platform health. **Needs:** A dashboard with actionable moderation queues and platform metrics. **Frustrations:** Manual moderation without tooling doesn't scale. **Desired outcome:** Low abuse rate, high trust, smooth dispute resolution. **Platform interaction:** Manages users, verifies organizations, resolves reports, reviews analytics.

07 — User Jobs-To-Be-Done

Resource Provider

- 1 **Functional job:** "Help me get this item to someone who needs it without extended back-and-forth."
- 1 **Emotional job:** "Let me feel confident my contribution actually helped someone."
- 1 **Social job:** "Let me be seen as a reliable, contributing member of my community."

JTBD statements:

- 1 When I have a surplus item, I want to list it quickly, so that I don't have to write a fresh post every time.
- 2 When someone requests my item, I want to see whether their need is genuine, so that I give it to the right person.
- 3 When a handover happens, I want to confirm it in the app, so that there is a record of the contribution.

Resource Seeker

- 1 **Functional job:** "Help me find a specific resource nearby before it's gone."
- 1 **Emotional job:** "Let me avoid the discomfort of repeatedly asking around."
- 1 **Social job:** "Let me access help in a dignified, low-friction way."

JTBD statements:

- 1 When I need an item, I want to publish a clear request, so that providers can find me instead of the reverse.
- 2 When a match appears, I want to be notified promptly, so that I don't miss a time-sensitive opportunity.
- 3 When I receive an item, I want to confirm it easily, so that the request closes and stops generating new matches.

Community Organization

- 1 **Functional job:** "Help me source resources reliably at scale."
- 1 **Emotional job:** "Let me trust that the platform represents us accurately."
- 1 **Social job:** "Let me demonstrate impact to our own stakeholders and donors."

JTBD statements:

- 1 When we have an ongoing need, I want to publish a standing request, so that we don't have to re-post constantly.
- 2 When we're new to the platform, I want a clear verification process, so that our organization is trusted by providers.
- 3 When a distribution period ends, I want a summary of what we received, so that we can report our impact.

08

THE TRANSFER LOOP

A good match is only the beginning; the product earns trust through follow-through.

08 — User Journey

Provider Journey

Discover → Register → Publish Resource → Match → Coordinate → Handover → Confirm → Impact

Stage	User Action	System Response	Pain Point	Opportunity	Trust Checkpoint
Discover	Learns about Dawwarha	Landing page explains value	Unfamiliarity	Clear, fast onboarding	—
Register	Creates account	Validates + creates profile	Friction if form is long	Minimal required fields	Email/basic verification
Publish	Lists a resource	Validates + publishes listing	Unsure what info to include	Guided form with examples	Listing review rules
Match	System proposes matches	Shows ranked candidate requests	Too many/few matches	Clear match scoring	—
Coordinate	Chooses a match, arranges handover	Provides contact/cordination info	Logistics uncertainty	In-app coordination status	Both sides see match details
Handover	Meets seeker, hands over item	Prompts confirmation	No-shows	Reminder notifications	Two-sided confirmation
Confirm	Confirms completion	Updates lifecycle to Completed	Forgetting to confirm	One-tap confirmation	Confirmation required from both sides
Impact	Views contribution record	Displays updated history	—	Visible contribution history	—

Seeker Journey

Need → Search / Request → Match → Accept → Receive → Confirm

Key pain points: outdated listings, uncertainty about legitimacy. Opportunities: real-time availability status, clear urgency signaling. Trust checkpoint: confirmation required before a request is marked fulfilled.

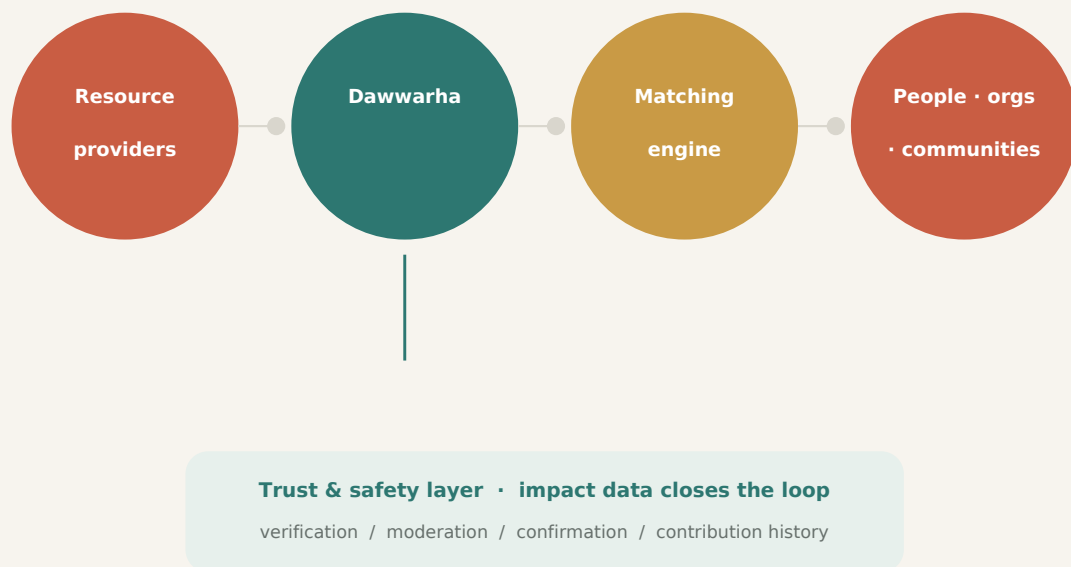
Organization Journey

Register → Verification → Publish Need → Receive Match → Coordinate → Confirm

Key pain point: verification delay before being able to act. Opportunity: a clear, transparent verification checklist. Trust checkpoint: admin-reviewed verification before organization-level requests go live.

09 — Product Ecosystem

A coordination layer between surplus, need, and proof.

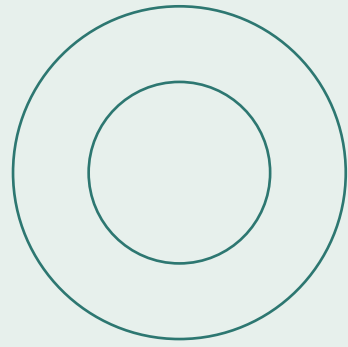


Dawwarha sits between two populations that already exist independently — people with surplus and people with need — and provides the coordination layer neither side has today. The matching engine converts raw listings and requests into ranked candidate connections. The trust & safety layer governs what enters the ecosystem and how disputes are resolved. Impact data closes the loop, informing both users (contribution history) and administrators (platform health).

10

THE PRODUCT SYSTEM

A focused MVP turns discovery, matching, handover, and confirmation into one loop.



10 — Core Features

P0 — Essential MVP

Feature	Purpose	User	Problem Solved	Expected Outcome
Authentication & roles	Secure, role-aware access	All	Unauthorized access	Trusted sessions
Resource CRUD	Publish/manage surplus listings	Provider	No structured way to list items	Discoverable listings
Request CRUD	Publish/manage needs	Seeker, Org	No structured way to ask for items	Discoverable requests
Rule-based matching	Suggest candidate connections	All	Manual searching	Faster, relevant matches
Resource lifecycle tracking	Track state from draft to completed	All	No accountability for outcomes	Verifiable transfers
Basic organization verification	Confirm organization legitimacy	Org, Admin	Trust risk with unverified orgs	Safer org-level requests
Reporting & moderation	Flag and remove bad content	All, Admin	Spam/fraud	Cleaner platform
Admin dashboard (basic)	Platform oversight	Admin	No visibility into platform health	Informed moderation

P1 — Important Post-MVP

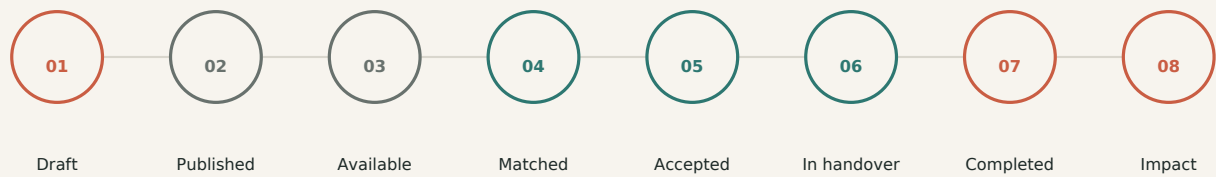
- 1 Notifications (email/in-app) for new matches and status changes.
- 1 Enhanced user reputation based on completed transfer history.
- 1 Organization-level standing requests (recurring needs).
- 1 Contribution history / impact profile page.

P2 — Future Expansion

- 1 Map-based discovery.
- 1 Smarter recommendation logic building on the rule-based engine.
- 1 Delivery/logistics coordination features.
- 1 Community campaigns tied to specific organizations or events.

11 — Resource Lifecycle

The transfer is a system, not a single event.



The product promise

A transfer is not complete until both sides confirm it.

State	Meaning
Draft	Listing created but not yet visible to others
Published	Visible in search/browse, not yet matched
Available	Actively open for matching
Matched	System or provider has proposed a candidate request
Accepted	Both sides agreed to proceed
In Handover	Coordination in progress, physical transfer pending
Completed	Both sides confirmed transfer occurred
Impact Recorded	Outcome logged against platform impact metrics

Edge cases:

- Resource expires:** listing auto-transitions to **Expired** after its availability window; provider is prompted to renew or archive.
- User cancels:** listing moves to **Cancelled**; any pending match is released back to **Available** pool for other requests.
- Request is rejected:** the specific match is discarded; the resource returns to **Available** and re-enters matching.
- Match fails (no-show, no response):** after a defined timeout, the match is released and the resource returns to **Available**; repeated failures are visible to moderation.
- Resource becomes unavailable:** provider can mark as **Unavailable** at any time, halting new matches without deleting listing history.
- User reports a problem:** listing/request is flagged for admin review; may be suspended pending resolution.

12 — Matching System

Matching is a flagship feature but is deliberately implemented as a **transparent, rule-based scoring engine** for the MVP — not machine learning — because a rule-based system is explainable, testable, fast to build within the NTI timeline, and sufficient for the actual problem (connecting nearby, compatible, time-relevant supply and demand).

Matching Signals

- Category match (resource type vs. request type)
- Location proximity / area match

- 1 Quantity compatibility (offered vs. requested amount)
- 1 Urgency of the request
- 1 Availability window of the resource
- 1 Organization-specific requirements (if applicable)

Simple Scoring Model

$$\text{Match score} = \text{category} \times W1 + \text{location} \times W2 + \text{quantity} \times W3 + \text{urgency} \times W4 + \text{availability} \times W5$$

Inputs are normalized, weighted, ranked, and thresholded.



Each sub-score is normalized (e.g., 0–1), weights (W1–W5) are configurable constants tuned during testing, and candidate matches are ranked by total score. A minimum threshold prevents low-quality matches from being surfaced.

Evolution Path

The rule-based engine is designed so its inputs and scoring logic are cleanly separated from the UI and API layer. This allows a future phase to replace or augment individual scoring components (e.g., a smarter urgency or location model) without redesigning the matching workflow — evolving toward a more intelligent recommendation system only when justified by real usage data (see Section 32).

13 — Trust & Safety

- 1 **Account verification:** basic email verification required for all users at registration.
- 1 **Organization verification:** organizations submit identifying information; admin reviews and approves before organization-level features (standing requests, verified badge) activate.
- 1 **Reporting:** any user can report a listing, request, or user profile.
- 1 **Moderation:** admin queue for reviewing reports, with the ability to suspend or remove content.
- 1 **Listing status:** suspicious or reported listings are automatically de-prioritized in matching pending review.
- 1 **Suspicious behavior detection (rule-based, MVP-appropriate):** simple heuristics such as repeated rapid listing creation, repeated failed handovers, or multiple reports trigger a review flag.
- 1 **User reputation:** derived from completed vs. failed/cancelled transfer history, visible to other users as a trust signal.
- 1 **Transaction confirmation:** both sides must confirm a handover before it counts as completed, preventing one-sided false claims.
- 1 **Admin intervention:** admins can suspend accounts, remove listings, and resolve disputes.
- 1 **Abuse prevention:** rate limiting on listing/request creation to reduce spam.

Sensitive categories: for categories such as food, the platform explicitly states applicable safety expectations (e.g., item condition disclosure, no expired perishables) as part of the listing flow, rather

than assuming unrestricted transfer. Legal responsibility for the condition and legality of transferred items rests with the transferring parties; Dawwarha provides coordination, not certification.

14 — Roles & Authorization

Roles

- 1 **User** — individual provider and/or seeker.
- 1 **Organization** — verified entity that can publish standing needs and receive resources at scale.
- 1 **Admin** — platform operator with moderation and verification authority.

Permissions Matrix

Capability	User	Organization	Admin
Register / Login	✓	✓	✓
Create Resource	✓	✓	✓
Create Request	✓	✓	✓
Manage Own Listings	✓	✓	✓
Publish Standing Request	—	✓	✓
Verify Organizations	—	—	✓
Moderate Content / Resolve Reports	—	—	✓
Suspend Users	—	—	✓
View Own Contribution History	✓	✓	✓
View Platform Analytics	—	Limited (own org)	✓ (full)

15 — CRUD Mapping

Resource

- 1 **Create:** Provider publishes a new resource listing.
- 1 **Read:** Any authenticated user browses/searches resources.
- 1 **Update:** Provider edits quantity, description, availability, or status.
- 1 **Delete:** Provider (or admin) removes a listing.

Request

- 1 **Create:** Seeker or organization publishes a need.
- 1 **Read:** Providers and admins browse open requests relevant to their listings.
- 1 **Update:** Requester edits urgency, quantity, or details.
- 1 **Delete:** Requester (or admin) withdraws a request.

Organization

- 1 **Create:** Organization registers and submits verification details.
- 1 **Read:** Public profile visible to providers considering a match.

- 1 **Update:** Organization updates its profile/contact/verification documents.
- 1 **Delete:** Admin can deactivate an organization for policy violations.

Categories

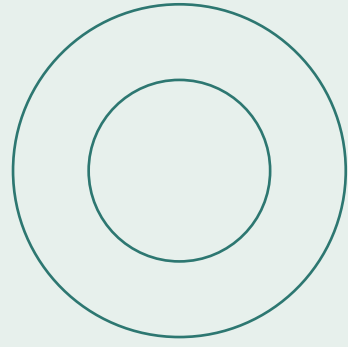
- 1 **Create/Update/Delete:** Admin manages the controlled category taxonomy (food, books, furniture, clothing, school supplies, office equipment, other).
- 1 **Read:** Used throughout resource/request forms and filters.

Each CRUD surface maps to a genuine product need (listing management, need management, organizational trust, and consistent categorization) rather than existing solely to satisfy a training requirement.

16

THE CONTRACT

Requirements make the product promise specific enough to build and test.



16 — Functional Requirements

FR-001 The system shall allow a new user to register with email, password, and basic profile info.
FR-002 The system shall allow a registered user to log in and receive an authenticated session.
FR-003 The system shall allow a user to log out and invalidate their session.
FR-004 The system shall enforce role-based access control for User, Organization, and Admin roles.
FR-005 The system shall allow a provider to create a resource listing with category, quantity, description, location, and availability window.
FR-006 The system shall allow a provider to update or delete their own resource listing.
FR-007 The system shall allow a seeker or organization to create a request with category, quantity, urgency, and location.
FR-008 The system shall allow a requester to update or delete their own request.
FR-009 The system shall compute a match score between open resources and open requests using the defined scoring model.
FR-010 The system shall present ranked candidate matches to both the provider and the requester.
FR-011 The system shall allow a provider or requester to accept or reject a proposed match.
FR-012 The system shall track a resource through its defined lifecycle states.
FR-013 The system shall require confirmation from both parties before marking a transfer as Completed.
FR-014 The system shall allow any authenticated user to report a listing, request, or profile.
FR-015 The system shall provide an admin interface to review and act on reports.
FR-016 The system shall allow an organization to submit verification information.
FR-017 The system shall allow an admin to approve or reject organization verification.
FR-018 The system shall display a contribution history for each user (resources given, requests fulfilled).
FR-019 The system shall provide an admin dashboard summarizing key platform metrics.
FR-020 The system shall validate all form inputs on both frontend and backend.
FR-021 The system shall return meaningful, user-facing error messages on failed operations.
FR-022 The system shall support filtering and searching resources/requests by category and location.
FR-023 The system shall allow a user to view and edit their own profile.
FR-024 The system shall manage a controlled category taxonomy administrable only by Admin.

17 — Non-Functional Requirements

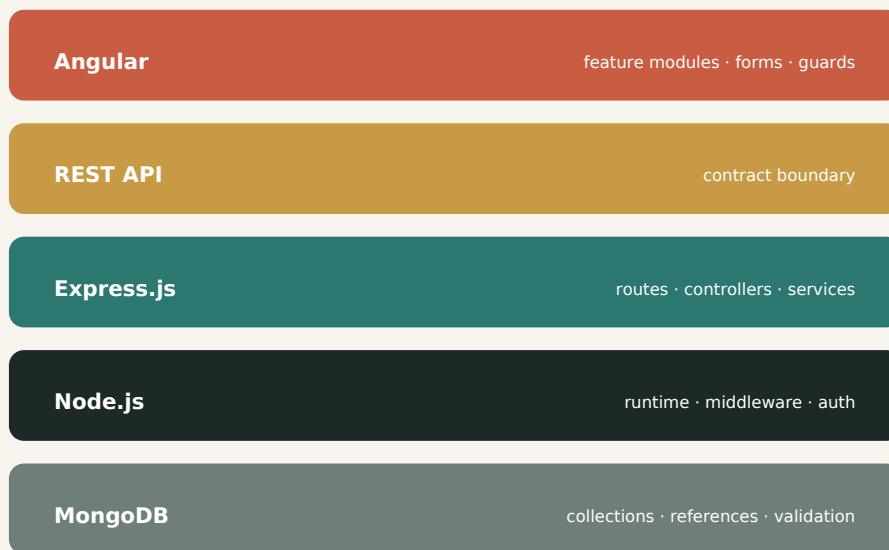
NFR-001 The system shall respond to standard API requests within 500ms under normal load (development/testing environment).
NFR-002 The system shall hash and securely store all user passwords.
NFR-003 The system shall protect all authenticated routes with token-based authorization.
NFR-004 The system shall validate and sanitize all user input to prevent injection attacks.
NFR-005 The system shall be usable on both desktop and mobile viewport sizes (responsive design).
NFR-006 The system shall provide loading indicators for all asynchronous operations exceeding 300ms.
NFR-007 The system shall provide clear success and error feedback for all user actions.
NFR-008 The system's codebase shall follow a modular, layered architecture (routes/controllers/services on the backend; components/services on the frontend) to support maintainability.
NFR-009 The system shall maintain data integrity through backend-enforced schema validation.
NFR-010 The system shall be structured to allow horizontal scaling of the Node.js API layer in a future production deployment.
NFR-011 The system shall follow basic accessibility practices (semantic HTML, sufficient color contrast, keyboard navigability for core flows).
NFR-012 The system shall log significant errors server-side for debugging purposes.

18

A modular MEAN architecture keeps the product explainable, testable, and ready to grow.

18 — MEAN Architecture

Technical direction



Frontend Architecture

- 1 **Components:** feature-based (resources, requests, matches, organizations, admin, auth, shared UI).
- 1 **Services:** encapsulate all HTTP communication with the backend API.
- 1 **Guards:** route guards enforce authentication and role-based access (e.g., AdminGuard, OrgGuard).
- 1 **Interceptors:** attach auth tokens to outgoing requests and handle global error responses.
- 1 **Reactive Forms & Validators:** all data-entry forms (resource, request, org verification) use Angular Reactive Forms with custom validators mirroring backend rules.
- 1 **State considerations:** feature-level services hold local state for MVP scope; a dedicated state management library is not required at this scale.
- 1 **Routing:** lazy-loaded feature modules per role area (public, user, organization, admin).

Backend Architecture

- 1 **Routes:** define REST endpoints grouped by resource domain.
- 1 **Controllers:** handle request/response, delegate business logic to services.
- 1 **Services:** contain core business logic (matching engine, lifecycle transitions, verification workflow).
- 1 **Middleware:** authentication, authorization, validation, centralized error handling.
- 1 **Validation:** schema-based request validation before controller logic executes.
- 1 **Authentication/Authorization:** JWT-based sessions; role checks enforced via middleware.
- 1 **Error handling:** centralized error-handling middleware returning consistent error response shapes.

Database

MongoDB collections are organized around the core entities in Section 19, with relationships expressed via references (ObjectId) rather than deep embedding, to keep documents maintainable as data grows.

19 — Data Model

Entity	Purpose	Key Fields	Relationships	Validation Notes
User	Represents an individual account	name, email, passwordHash, role, location, createdAt	Has many Resources, Requests	Unique email; required role
Organization	Represents a verified entity	name, description, contactInfo, verificationStatus, documents, ownerId	Belongs to a User (owner); has many Requests	verificationStatus enum controlled by Admin
Resource	A listed surplus item	title, category, quantity, description, location, availabilityWindow, status, providerId	Belongs to User/Organization; linked to Matches	status enum follows lifecycle in Section 11
Request	A published need	category, quantity, urgency, location, status, requesterId	Belongs to User/Organization; linked to Matches	urgency enum (low/medium/high); status enum
Match	A proposed connection between Resource and Request	resourceId, requestId, score, status, createdAt	References Resource and Request	status enum (proposed/accepted/rejected/expired)
Transaction / Handover	Record of a physical transfer	matchId, confirmedByProvider, confirmedBySeeker, completedAt	References Match	Completed only when both confirmations are true
Category	Controlled taxonomy	name, description	Referenced by Resource, Request	Admin-managed only
Report	A flag on content or a user	reporterId, targetType, targetId, reason, status	References User, Resource/Request/User	status enum (open/reviewed/resolved)
Notification	User-facing alert	userId, type, message, read, createdAt	Belongs to User	—

No unnecessary fields (e.g., speculative payment or logistics fields) are included in the MVP schema; these are deferred to future phases per Section 32.

20 — API Design

```

POST /api/auth/register
POST /api/auth/login
POST /api/auth/logout

GET /api/users/me
PUT /api/users/me

GET /api/resources
POST /api/resources
GET /api/resources/:id
PUT /api/resources/:id
DELETE /api/resources/:id

GET /api/requests
POST /api/requests
GET /api/requests/:id
PUT /api/requests/:id
DELETE /api/requests/:id

```

```
GET /api/matches
POST /api/matches/:resourceId/generate
PUT /api/matches/:id/accept
PUT /api/matches/:id/reject

POST /api/transactions/:matchId/confirm

POST /api/organizations
GET /api/organizations/:id
PUT /api/organizations/:id
POST /api/organizations/:id/verify (Admin)

POST /api/reports
GET /api/reports (Admin)
PUT /api/reports/:id/resolve (Admin)

GET /api/categories
POST /api/categories (Admin)
PUT /api/categories/:id (Admin)
DELETE /api/categories/:id (Admin)

GET /api/admin/analytics (Admin)
```

Endpoints are grouped by resource domain, with role-restricted endpoints clearly marked. No implementation code is specified here; this is a contract-level map for backend and frontend teams to build against in parallel.

21 — Validation Strategy

Angular (Frontend)

Reactive Forms with synchronous validators (required fields, string length, numeric ranges, enum values) and async validators where needed (e.g., email uniqueness pre-check). Frontend validation exists to give users immediate feedback and reduce unnecessary API calls.

Express / Backend

Every write endpoint validates the incoming payload against a defined schema before it reaches business logic. Backend validation is **mandatory even when frontend validation exists** because the frontend can be bypassed entirely (direct API calls, modified clients, malicious actors) — the backend is the only enforcement point that can be trusted.

Examples:

- 1 Required fields: resource category and quantity cannot be empty.
- 1 Quantity: must be a positive number.
- 1 Dates: availabilityWindow end date must be after start date.
- 1 Text length: descriptions capped to a reasonable maximum to prevent abuse.
- 1 Invalid values: category must match the controlled taxonomy, not free text.
- 1 Duplicate accounts: registration rejected if email already exists.
- 1 Unauthorized operations: a user attempting to edit another user's resource receives a 403, enforced server-side regardless of what the UI allows.

22 — Error Handling

Error Type	Handling Approach	User-Facing Message Example
Validation errors	Return 400 with field-level details	"Quantity must be greater than 0."
Authentication errors	Return 401, redirect to login	"Please log in to continue."
Authorization errors	Return 403	"You don't have permission to do that."
Not found	Return 404	"This listing no longer exists."
Conflict	Return 409	"This request has already been matched."
Server errors	Return 500, log internally, generic message externally	"Something went wrong. Please try again."
Network failures	Frontend retry/backoff with clear offline indicator	"Connection lost — retrying..."

A centralized Express error-handling middleware ensures all errors are caught, logged consistently, and returned in a uniform response shape (`{ success, message, errors? }`), which the Angular interceptor layer maps to consistent UI feedback.

THE EXPERIENCE

Clarity over decoration: every screen should make the next action obvious.

23 — UX & UI Principles

Design Principles

Clarity over decoration. Every screen answers "what can I do here, and what happens if I do it."
Community-oriented visual tone (warm but not childish), consistent with a platform people trust to handle real transfers.

Navigation Model

Role-aware navigation: public users see browse/discover; authenticated users see a dashboard-centric structure (My Resources, My Requests, Matches, Profile); organizations see an additional org-specific section; admins see a separate moderation-focused shell.

Information Hierarchy

Listings lead with category, quantity, and location — the three facts a user needs to decide relevance — before secondary description text.

Empty States

Every list view (My Resources, Matches, Requests) has a purposeful empty state with a direct call to action, not a blank screen.

Loading States

Skeleton or spinner feedback for any operation over ~300ms, applied consistently across list and form views.

Error States

Inline field errors on forms; toast/banner notifications for action-level failures, per Section 22.

Success States

Explicit confirmation on completed actions (listing published, match accepted, transfer confirmed) — never a silent success.

Accessibility

Semantic HTML structure, sufficient contrast ratios, labeled form controls, and keyboard-operable primary flows (registration, listing creation, match acceptance).

24 — Key Screens

Public

Screen	Purpose	Main Content	Primary Action	Important States
Landing Page	Explain value proposition	Hero, how-it-works, category highlights	Register / Browse	—
About	Explain mission and model	Vision, trust approach	—	—
Browse Resources	Discover available items	Filterable listing grid	Request/contact provider	Empty, loading, filtered
Browse Requests	Discover community needs	Filterable request list	Offer a resource	Empty, loading, filtered
Login / Register	Authenticate	Forms	Submit	Validation errors

User

Screen	Purpose	Main Content	Primary Action	Important States
Dashboard	Overview of activity	Active listings/requests, recent matches	Navigate to detail	Empty (new user)
Create Resource	Publish surplus item	Guided form	Publish	Validation errors
My Resources	Manage own listings	List with status	Edit/delete/mark unavailable	Empty
My Requests	Manage own requests	List with status	Edit/withdraw	Empty
Matches	Review proposed connections	Ranked candidates	Accept/reject	Empty, pending
Notifications	View alerts	Chronological list	Mark read	Empty
Profile	Manage account	Editable profile fields	Save	Validation errors
Contribution History	View impact	Completed transfers	—	Empty

Organization

Screen	Purpose	Main Content	Primary Action	Important States
Organization Dashboard	Overview of org activity	Standing requests, matches	Navigate	Empty
Verification	Submit/track verification	Document upload, status	Submit	Pending/approved/rejected
Requests	Manage org needs	List with status	Create/edit	Empty
Received Resources	Track incoming matches	List with confirmation status	Confirm receipt	Pending
Impact	View aggregate outcomes	Summary metrics	Export (future)	Empty

Admin

Screen	Purpose	Main Content	Primary Action	Important States
Admin Dashboard	Platform health overview	Key metrics (Section 25)	Drill into detail	—
Users	Manage accounts	Searchable list	Suspend/reactivate	—
Organizations	Manage verification	Pending/approved list	Approve/reject	Pending
Resources / Requests	Oversight of listings	Searchable list	Remove/flag	—
Reports	Moderation queue	Open reports	Resolve	Open/resolved
Categories	Taxonomy management	List	Add/edit/remove	—
Analytics	Platform-wide metrics	Charts/tables	—	—

25 — Admin Dashboard

Conceptual dashboard view — metrics are definitions, not fabricated values.

Possible metrics: Total Users, Active Users, Resources Published, Resources Rescued (Completed), Requests Created, Requests Fulfilled, Successful Matches, Registered Organizations, Open Reports, Recent Platform Activity.

Decisions the dashboard enables:

- 1 Whether moderation capacity needs to increase (open reports trending up).
- 1 Whether matching quality is adequate (match acceptance rate).
- 1 Whether organization onboarding is a bottleneck (pending verification backlog).
- 1 Whether specific categories are undersupplied relative to demand (published vs. fulfilled by category).

26

THE MEASURE

Count what the platform can prove: completed transfers, fulfilled needs, and trust.

26 — Impact Measurement

No environmental or monetary impact figures are calculated in the MVP, since no verified methodology exists yet for this project. All metrics below are platform-native and directly measurable.

North Star Metric

Completed Transfers — the number of resource-to-request matches confirmed as completed by both parties. This single metric captures the platform's actual purpose: successfully moving a resource from surplus to need.

Activation Metrics

- 1 % of registered users who publish at least one resource or request within their first session.

Engagement Metrics

- 1 Weekly active listers/requesters.
- 1 Average listings per active provider.

Matching Metrics

- 1 Match proposal rate (matches generated per open listing).
- 1 Match acceptance rate.

Success Metrics

- 1 Completed Transfers (North Star).
- 1 Request fulfillment rate.

Retention Metrics

- 1 % of users returning to publish a second listing/request within 30 days.

Trust & Safety Metrics

- 1 Reports filed per 100 transactions.
- 1 Average time to report resolution.

Impact Metrics

- 1 Cumulative Completed Transfers.
- 1 Distinct organizations actively receiving matched resources.

Each metric is chosen because it is directly observable from platform data, without requiring external assumptions.

27 — Business / Sustainability Model

Current Project Scope: Dawwarha is developed as a non-commercial NTI graduation project. No payment processing, subscription, or monetization logic is part of the MVP.

Future Business Possibilities (Hypothesis, not committed):

- 1 NGO and institutional partnerships (formal onboarding of established organizations).
- 1 CSR-sponsored community campaigns run through the platform.
- 1 Optional premium tools for organizations managing high volumes of requests (e.g., advanced reporting).

- 1 Logistics partnerships to support handover in cases where self-coordinated transfer is impractical.

These are explicitly future explorations and are not required for, or assumed within, the graduation MVP.

28 — Competitive Landscape

Dimension	Facebook Groups	WhatsApp Groups	Classified Platforms	Traditional Donation Orgs	Dawwarha
Resource-specific focus	No	No	Partial (buy/sell)	Yes	Yes
Structured need publishing	No	No	No	Manual	Yes
Matching logic	No	No	No	Manual	Yes (rule-based)
Location awareness	Limited	No	Partial	Manual	Yes
Verification	No	No	Limited	Yes (organizational)	Yes (org verification)
Lifecycle / outcome tracking	No	No	No	Manual/internal	Yes
Community contribution visibility	No	No	No	Internal only	Yes

Dawwarha's differentiation is not novelty of concept but **structural completeness**: it is the only option in this comparison that combines need publishing, location- and urgency-aware matching, a defined transfer lifecycle, and verifiable outcomes in one coordinated system. No claim is made that Dawwarha is the first platform of its kind globally; the differentiation claim is scoped to the alternatives realistically available to its target communities today.

29 — SWOT Analysis

Strengths

- 1 Addresses a real, structurally-defined coordination gap.
- 1 Clear, testable MVP scope achievable by a 4-person team on the MEAN stack.
- 1 Trust and lifecycle mechanisms differentiate it from informal channels.

Weaknesses

- 1 Cold-start problem: value depends on having both providers and seekers present simultaneously.
- 1 No dedicated logistics/delivery layer in the MVP; transfer still relies on self-coordination.
- 1 Small initial team limits moderation capacity at scale.

Opportunities

- 1 Partnership potential with local NGOs and schools for organized distribution.
- 1 Expansion into new resource categories and geographies over time.
- 1 Foundation for future intelligent matching once usage data accumulates.

Threats

- 1 Low initial adoption if either supply or demand side fails to reach critical mass.

- 1 Reputation risk from a single high-profile fraud or safety incident if trust mechanisms are inadequate.
- 1 Competing informal habits (existing Facebook/WhatsApp groups) that users are reluctant to abandon.

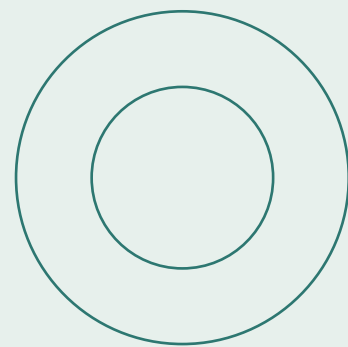
30 — Risks & Mitigation

Risk	Probability	Impact	Mitigation
Fake listings	Medium	Medium	Reporting system, reputation scoring, admin review
Abuse / harassment between users	Low	Medium	Reporting, admin suspension authority
Food safety issues	Medium	High	Explicit safety disclosures in listing flow; disclaim platform responsibility for item condition
Transportation / handover failure	High	Low	Location-aware matching, no-show tracking, match re-release
Low adoption (cold start)	High	High	Seed initial listings/requests through a pilot community or organization partnership
Poor-quality data (vague listings)	Medium	Medium	Guided, validated listing forms with required structured fields
Privacy exposure	Low	Medium	Minimal required personal data, contact info shared only after match acceptance
Technical complexity exceeding timeline	Medium	High	Strict MVP scope discipline (Section 31), defer P1/P2 features
Moderation workload exceeding capacity	Medium	Medium	Rule-based flagging to prioritize admin attention, not manual scanning of all content

31

THE PATH TO DELIVERY

Scope discipline is the mechanism that turns a strong concept into a working graduation project.



31 — MVP Definition

The graduation-project MVP must deliver, in priority order:

- 1 Authentication
- 2 Authorization (role-based access control)
- 3 Resource CRUD
- 4 Request CRUD
- 5 Rule-based matching
- 6 User and organization profiles
- 7 Admin moderation (reports + organization verification)
- 8 Frontend and backend validation
- 9 Consistent error handling
- 10 Responsive UI across core flows
- 11 Basic impact/admin dashboard

Everything in Sections 10 (P1/P2), 27, and 32 is explicitly secondary and out of scope for the graded deliverable.

32 — Future Roadmap

Phase 1 — MVP

Core listing, request, matching, lifecycle, and moderation functionality as defined in Section 31.

Phase 2 — Community Growth

Notifications, reputation system refinement, organization standing requests, contribution history/profile pages.

Phase 3 — Intelligent Matching

Evolve the scoring engine using accumulated match/outcome data; explore smarter recommendation logic where genuinely justified by evidence, not by default.

Phase 4 — Institutional Partnerships

Formal NGO/school onboarding flows, campaign tools, reporting exports for partner organizations.

Phase 5 — Geographic Expansion

Map-based discovery, multi-region category taxonomies, localized moderation.

Potential future capabilities include maps, delivery coordination, advanced analytics, a mobile application, and AI-assisted categorization — none of which are mandatory, and each of which should be evaluated against real usage evidence before being built.

33 — Team Structure

The project team consists of 4 members. Proposed division, optimized to avoid isolated silos and ensure shared ownership of the integrated product:

Member 1 — Frontend Lead (Angular)

Owns core user-facing modules (resources, requests, browse/search), component architecture, and reactive forms/validation.

Member 2 — Backend Lead (Node.js / Express)

Owns API structure, controllers, middleware, and the matching engine's business logic.

Member 3 — Data & Auth Lead (MongoDB / Authentication)

Owns schema design, authentication/authorization implementation, and API security.

Member 4 — UX, Admin & Integration Lead

Owns admin dashboard, organization verification flow, cross-module integration testing, and end-to-end UX consistency.

All members participate in code review, and the matching engine and lifecycle logic (the product's core differentiator) are treated as a shared responsibility between Members 2 and 3 rather than owned by a single person, ensuring no single point of failure on the most critical feature.

34 — Development Strategy

- 1 **Git workflow:** trunk-based development with short-lived feature branches (feature/resource-crud, feature/matching-engine).
- 1 **Branch strategy:** main (stable), feature branches merged via Pull Request only.
- 1 **Pull Requests:** required for all merges to main; minimum one reviewer.
- 1 **Code reviews:** focused on correctness, validation coverage, and adherence to the architecture in Section 18.
- 1 **Environment configuration:** .env-based configuration for local/dev; no secrets committed to source control.
- 1 **API documentation:** maintained alongside the API map in Section 20 (e.g., a shared Postman collection or lightweight OpenAPI spec).
- 1 **Testing:** integrated into the PR process per Section 35.
- 1 **Integration strategy:** backend endpoints stubbed early so frontend and backend workstreams can proceed in parallel.
- 1 **Issue tracking:** a shared board (e.g., GitHub Projects/Issues) mapped to the MVP feature list in Section 31.

35 — Testing Strategy

Unit Testing

Core business logic (matching score calculation, lifecycle transitions) covered by isolated unit tests.

API Testing

Each endpoint in Section 20 tested for expected success and failure responses (e.g., Postman/automated test suite).

Frontend Testing

Component-level tests for critical forms (resource creation, request creation) verifying validation behavior.

Integration Testing

End-to-end flow tests covering: register → publish resource → publish request → match → accept → confirm.

Validation Testing

Explicit test cases for each validation rule in Section 21 (missing fields, invalid values, boundary cases).

Authorization Testing

Verify that role-restricted endpoints (e.g., admin-only routes) correctly reject unauthorized roles.

Responsive Testing

Manual and/or automated checks across common breakpoints (mobile, tablet, desktop) for all key screens in Section 24.

User Acceptance Testing

Structured walkthroughs of the Provider, Seeker, and Organization journeys (Section 08) with test users prior to demo.

Representative test scenarios:

- 1 A provider publishes a resource with missing quantity → backend rejects with a clear validation error.
- 1 A seeker attempts to edit another user's request → backend returns 403.
- 1 A match is accepted by only one party → transaction remains "In Handover," not "Completed."
- 1 An unverified organization attempts to publish a standing request → blocked until verification is approved.

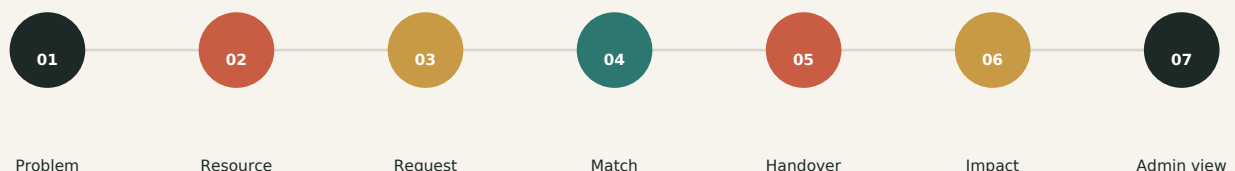
36 — Definition of Done

A feature is considered complete only when:

- 1 Functional behavior works as specified in its Functional Requirement(s).
- 1 UI is responsive across target breakpoints.
- 1 Frontend and backend validation are both implemented.
- 1 Errors are handled per Section 22 and surfaced with a meaningful message.
- 1 Authorization is enforced server-side, not just hidden in the UI.
- 1 API behavior is covered by at least a basic test.
- 1 Code has been reviewed by at least one other team member.
- 1 No critical or blocking bugs remain open against the feature.

37 — Demo Strategy

Target duration: 5–7 minutes, structured as a narrative rather than a menu-by-menu walkthrough.



Opening (0:00-0:45): State the problem in one sentence and show the fragmented-channel reality (a mock Facebook-group screenshot or description) before switching to Dawwarha — the product's value

should be obvious within the first 60 seconds.

Middle (0:45-5:30): Live walkthrough following the narrative above, using one pre-seeded provider account and one pre-seeded seeker/organization account to show the full lifecycle without dead time.

Close (5:30-7:00): Show the admin dashboard reflecting the just-completed transfer, then close on the final positioning statement (Section 40).

38 — Evaluation Strategy

NTI Requirement	Dawwarha Implementation
MEAN Stack	Angular frontend, Express/Node.js API, MongoDB data layer (Section 18)
Authentication	JWT-based register/login/logout (FR-001-FR-003)
Authorization	Role-based access control for User/Organization/Admin (FR-004, Section 14)
CRUD	Resource, Request, Organization, Category CRUD (Section 15)
Validation	Frontend Reactive Forms + backend schema validation (Section 21)
Error Handling	Centralized error middleware, consistent response shape (Section 22)
Responsive UI	Mobile-first layouts across all key screens (Section 24)
Clean Structure	Layered backend (routes/controllers/services/middleware) and modular frontend (components/services/guards) (Section 18)
Loading States	Defined loading indicators for async operations (NFR-006)
Success/Error Messages	Standardized feedback patterns across all flows (Section 23)

39 — Product Success Definition

"How will we know Dawwarha actually works?"

Success is defined as a functioning progression through:

Adoption → users register and publish real listings/requests **Matching** → the scoring engine surfaces relevant, ranked candidate connections **Completion** → matches convert into two-sided confirmed transfers **Impact** → completed transfers accumulate into a visible, trustworthy record **Trust** → the reporting and verification systems keep abuse low enough that users keep returning

If a meaningful number of transfers move end-to-end through this chain — not just listings created, but transfers confirmed by both sides — the product is working as intended.

40 — Final Product Positioning

Dawwarha is not simply a website for donating unused things.

It is a **community resource coordination and recovery platform** — infrastructure for making sure that when someone has what someone else needs, distance and disconnection are no longer the reason it

goes to waste.

دورها. Someone, somewhere, needs it. Let's help it find them.

Prepared as the foundational product concept document for the Dawwarha graduation project — intended to serve as the single source of truth from which the PRD, SRS, UX/UI specifications, and technical architecture will be derived.